

# QuizPulse: System Design & Engineering Documentation

## 1. Executive Summary

QuizPulse is a scalable, real-time, full-stack quiz platform designed to facilitate interactive multiplayer sessions. The architecture leverages a modular design, separating core business logic from transport layers. Key features include AI-driven content generation from text and PDF sources, real-time state synchronization, and a clean, feature-first backend organization.

## 2. High-Level System Architecture

The system follows a traditional client-server architecture with persistent WebSocket connections for real-time interactivity. The stack consists of:

| Layer     | Technology        |
|-----------|-------------------|
| Frontend  | React, TypeScript |
| Backend   | Node.js, Express  |
| Real-Time | Socket.IO         |
| Database  | MongoDB (Atlas)   |
| AI Engine | Google Gemini API |

## 3. Backend Design Principles

The backend was engineered for maintainability and scalability, adhering to the following paradigms:

- **Feature-Based Modularization:** Business domains (AI, Quiz, Room, Game) are isolated into independent feature modules, each containing its own controllers, services, models, and routes.
- **Layered Architecture:** Separation of concerns is enforced via a distinct layering strategy:
  - **Controller Layer:** Handles HTTP ingress, request validation, and response egress.
  - **Service Layer:** Contains centralized business logic and orchestrates data flow.
  - **Model Layer:** Represents persistent data structures via Mongoose schemas.
- **Single Responsibility Principle (SRP):** Every module and service is scoped to a singular function, facilitating unit testing and rapid debugging.

## 4. Real-Time Communication Design

Socket.IO is utilized to manage state synchronization across distributed clients. Communication is isolated using Socket "rooms," ensuring that quiz events are scoped to specific sessions. Key events include:

- Player Joined / Quiz Started
- Question Broadcast / Answer Submitted
- Leaderboard Updated / Quiz Finished

## 5. Development & Deployment Strategy

- **TypeScript:** Enforced type safety across the stack to minimize runtime errors.
- **Environment Management:** Centralized configuration for cross-environment consistency (Vercel for frontend, Render for backend).
- **Error Handling:** Standardized try-catch blocks to ensure consistent JSON response formats during API failures.

## 6. Future Extensibility

The current architecture supports future expansion through:

- **Performance:** Implementation of Redis for low-latency session caching.
- **Scalability:** Future transition to Kubernetes-based container orchestration.
- **Features:** Integration of Role-Based Access Control (RBAC), advanced quiz analytics, and CI/CD automation.